

ASSIGNMENT

Course: Full Stack Web Development

Course Code: 23CM4121

Assignment No.: 1

Student Name: T. Akshaya

Roll No.: A24126552122

1. TITLE

Implement Inheritance Types using JavaScript

2. AIM

To implement and demonstrate the different types of inheritance supported in JavaScript using ES6 classes, and to understand how objects and methods are shared and extended across a class hierarchy.

3. OBJECTIVE

- To understand the concept of inheritance in Object-Oriented Programming using JavaScript.
 - To implement Simple (Single) Inheritance using the extends keyword.
 - To implement Multilevel Inheritance by extending a chain of classes.
 - To implement Hierarchical Inheritance where multiple child classes extend a single parent class.
 - To implement Multiple Inheritance in JavaScript using the mixin pattern, since classes cannot directly extend more than one class.
 - To implement Hybrid Inheritance by combining Hierarchical and Multiple Inheritance.
 - To use the super keyword and constructor chaining to access parent class properties.
-

4. THEORY

Inheritance in JavaScript

Inheritance is an Object-Oriented Programming concept that allows a class (child/derived class) to acquire the properties and methods of another class (parent/base class). In JavaScript (ES6 and later), inheritance is implemented using the class and extends keywords, which provide a cleaner syntax over the older prototype-based inheritance model.

Simple Inheritance

Simple (or single) inheritance occurs when one child class extends exactly one parent class, inheriting all of its properties and methods directly.

Multilevel Inheritance

Multilevel inheritance occurs when a class is derived from a class that is itself derived from another class, forming a chain (Grandparent -> Parent -> Child).

Hierarchical Inheritance

Hierarchical inheritance occurs when multiple child classes extend the same single parent class, each inheriting the parent's properties independently.

Multiple Inheritance (via Mixins)

JavaScript classes cannot directly extend more than one class. Multiple inheritance is instead achieved using the mixin pattern, where a function takes a base class and returns a new subclass extended with additional functionality. Several mixins can be composed together to combine behaviours from multiple sources into a single class.

Hybrid Inheritance

Hybrid inheritance is a combination of two or more types of inheritance, such as Hierarchical and Multiple inheritance used together in the same class design.

5. ALGORITHM / PROCEDURE

1. Create a new JavaScript file for each type of inheritance.
 2. Define a base (parent) class using the class keyword with a constructor and one or more methods.
 3. Create a child class using the extends keyword to inherit from the parent class.
 4. For multilevel inheritance, extend the child class further into a grandchild class.
 5. For hierarchical inheritance, create two or more child classes that each extend the same parent class.
 6. For multiple inheritance, define mixin functions that accept a base class and return an extended class.
 7. Compose multiple mixins together and extend the resulting class into a final class.
 8. For hybrid inheritance, combine mixins with a shared base class to merge multiple inheritance types.
 9. Create objects of the derived classes and call both inherited and own methods.
 10. Run each file using Node.js (node filename.js) and observe the console output.
 11. Record the output and verify that inherited properties and methods work as expected.
-

6. PROGRAM

simple.js — Simple (Single) Inheritance

```
class Animal {  
  constructor(name) {  
    this.name = name;  
  }  
  eat() {
```

```
    console.log(`${this.name} is eating.`);
  }
}

class Dog extends Animal {
  bark() {
    console.log(`${this.name} is barking.`);
  }
}

const d1 = new Dog("puppy");
d1.eat();
d1.bark();
```

multilevel.js — Multilevel Inheritance

```
class Animal {
  constructor(name) {
    this.name = name;
  }
  eat() {
    console.log(`${this.name} eats food.`);
  }
}

class Mammal extends Animal {
  walk() {
    console.log(`${this.name} walks on land.`);
  }
}

class Dog extends Mammal {
  bark() {
    console.log(`${this.name} barks.`);
  }
}

const d1 = new Dog("Tommy");
d1.eat();
d1.walk();
d1.bark();
```

hierarchical.js — Hierarchical Inheritance

```
class Vehicle {
  constructor(brand) {
    this.brand = brand;
  }
  start() {
    console.log(`${this.brand} vehicle started.`);
  }
}
```

```

    }
  }

  class Car extends Vehicle {
    drive() {
      console.log(`${this.brand} car is driving.`);
    }
  }

  class Bike extends Vehicle {
    ride() {
      console.log(`${this.brand} bike is riding.`);
    }
  }

  const car = new Car("Toyota");
  car.start();
  car.drive();

  const bike = new Bike("Honda");
  bike.start();
  bike.ride();

```

multiple.js — Multiple Inheritance (Mixins)

```

// JavaScript classes can't extend more than one class directly,
// so mixins are used.
let CanFly = Base => class extends Base {
  fly() {
    console.log(`${this.name} can fly.`);
  }
};

let CanSwim = Base => class extends Base {
  swim() {
    console.log(`${this.name} can swim.`);
  }
};

class Bird {
  constructor(name) {
    this.name = name;
  }
}

class Duck extends CanSwim(CanFly(Bird)) {}

const duck = new Duck("Donald");
duck.fly();

```

```
duck.swim();
```

hybrid.js — Hybrid Inheritance (Hierarchical + Multiple)

```
class Person {
  constructor(name) {
    this.name = name;
  }
  introduce() {
    console.log(`I am ${this.name}.`);
  }
}

let CanCode = Base => class extends Base {
  code() {
    console.log(`${this.name} writes code.`);
  }
};

let CanDesign = Base => class extends Base {
  design() {
    console.log(`${this.name} designs UI.`);
  }
};

class Developer extends CanDesign(CanCode(Person)) {
  work() {
    console.log(`${this.name} is working on a project.`);
  }
}

const dev = new Developer("Akshaya");
dev.introduce();
dev.code();
dev.design();
dev.work();
```

7. CODE EXPLANATION

Tag / Function	Purpose
class	Defines a blueprint (constructor and methods) for creating objects.
extends	Establishes inheritance, allowing a child class to acquire a parent class's properties and methods.

Tag / Function	Purpose
constructor()	Special method that initializes an object's properties when it is created with new.
this.name	Instance property storing the object's own data, accessible by all inherited methods.
super (implicit via extends)	Underlies the parent-child link so a subclass automatically has access to the parent's members.
Mixin function (Base => class ...)	Takes a base class and returns a new subclass extended with extra methods, used to simulate multiple inheritance.
Function composition (CanSwim(CanFly(Bird)))	Chains multiple mixins together so a single class inherits behaviour from more than one source.
new ClassName()	Creates a new object instance of the given class, running its constructor.
console.log()	Prints the result of a method call to the terminal for verification.

8. TEST CASES AND EXECUTION DATA

Test Case	Action	Expected Result	Actual Result	Status
TC01	Run simple.js	Dog object inherits eat() from Animal and uses its own bark()	puppy is eating. / puppy is barking.	Pass
TC02	Run multilevel.js	Dog inherits eat() from Animal and walk() from Mammal, plus its own bark()	Tommy eats food. / Tommy walks on land. / Tommy barks.	Pass
TC03	Run hierarchical.js	Car and Bike both inherit start() from Vehicle and add their own method	Toyota vehicle started. / Toyota car is driving. / Honda vehicle started. / Honda bike is riding.	Pass
TC04	Run multiple.js	Duck gains fly() and swim() from two separate mixins on Bird	Donald can fly. / Donald can swim.	Pass
TC05	Run hybrid.js	Developer combines Person, CanCode and	I am Akshaya. / Akshaya writes code. / Akshaya	Pass

Test Case	Action	Expected Result	Actual Result	Status
		CanDesign mixins with its own work()	designs UI. / Akshaya is working on a project.	

9. OUTPUT

Output 1: Simple Inheritance (simple.js)

```

Problems  Output  Debug Console  Terminal  Ports
PS C:\Users\syama\OneDrive\Desktop\122> cd Assignment
PS C:\Users\syama\OneDrive\Desktop\122\Assignment> node .\simple.js
puppy is eating.
puppy is barking.
PS C:\Users\syama\OneDrive\Desktop\122\Assignment> 

```

Output 2: Multilevel Inheritance (multilevel.js)

```

Problems  Output  Debug Console  Terminal  Ports
PS C:\Users\syama\OneDrive\Desktop\122\Assignment> node .\multilevel.js
Tommy eats food.
Tommy walks on land.
Tommy barks.
PS C:\Users\syama\OneDrive\Desktop\122\Assignment> 

```

Output 3: Hierarchical Inheritance (hierarchical.js)

```

PS C:\Users\syama\OneDrive\Desktop\122\Assignment> node .\hierarchical.js
Toyota vehicle started.
Toyota car is driving.
Honda vehicle started.
Honda bike is riding.
PS C:\Users\syama\OneDrive\Desktop\122\Assignment> 

```

Output 4: Multiple Inheritance (multiple.js)

```

Problems  Output  Debug Console  Terminal  Ports
PS C:\Users\syama\OneDrive\Desktop\122\Assignment> node .\multiple.js
Donald can fly.
Donald can swim.
PS C:\Users\syama\OneDrive\Desktop\122\Assignment> 

```

Output 5: Hybrid Inheritance (hybrid.js)

```
PS C:\Users\syama\OneDrive\Desktop\122\Assignment> node .\hybrid.js
I am Akshaya.
Akshaya writes code.
Akshaya designs UI.
Akshaya is working on a project.
PS C:\Users\syama\OneDrive\Desktop\122\Assignment> █
```

10. RESULT

Thus, the different types of inheritance in JavaScript — Simple, Multilevel, Hierarchical, Multiple (via mixins), and Hybrid — were successfully implemented using ES6 classes and the extends keyword. Each program was executed using Node.js, and the outputs verified that child classes correctly inherited and extended the properties and methods of their parent classes.